

深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： 回溯法——地图填色问题

学院： 计算机与软件学院

专业： 计算机科学与技术高性能班

指导教师： 杨烜

报告人： 姚泽棉 学号： 2024150026

实验时间： 2026年04月24日

实验报告提交时间： 2026年05月08日

教务部制

一、实验目的

- (1) 掌握回溯法算法设计思想。
- (2) 掌握地图填色问题的回溯法解法。

二、背景知识

为地图或其他由不同区域组成的图形着色时，相邻国家/地区不能使用相同的颜色。我们可能还想使用尽可能少的不同颜色进行填涂。一些简单的“地图”（例如棋盘）仅需要两种颜色（黑白），但是大多数复杂的地图需要更多颜色。

每张地图包含四个相互连接的国家时，它们至少需要四种颜色。1852 年，植物学专业的学生弗朗西斯·古思里（Francis Guthrie）于 1852 年首次提出“四色问题”。他观察到四种颜色似乎足以满足他尝试的任何地图填色问题，但他无法找到适用于所有地图的证明。这个问题被称为四色问题。长期以来，数学家无法证明四种颜色就够了，或者无法找到需要四种以上颜色的地图。直到 1976 年德国数学家沃尔夫冈·哈肯（Wolfgang Haken）（生于 1928 年）和肯尼斯·阿佩尔（Kenneth Appel，1932 年-2013 年）使用计算机证明了四色定理，他们将无数种可能的地图缩减为 1936 种特殊情况，每种情况都由一台计算机进行了总计超过 1000 个小时的检查。

他们因此工作获得了美国数学学会富尔克森奖。在 1990 年，哈肯（Haken）成为伊利诺伊大学（University of Illinois）高级研究中心的成员，他现在是该大学的名誉教授。

四色定理是第一个使用计算机证明的著名数学定理，此后变得越来越普遍，争议也越来越小。更快的计算机和更高效的算法意味着今天您可以在几个小时内笔记本电脑上证明四色颜色定理。

三、问题描述

- 1、对下面这个小规模数据，利用四色填色测试算法的正确性；



- 2、对附件中给定的三个地图数据尝试分别使用 5 个（le450_5a），15 个（le450_15b），25 个（le450_25a）颜色为地图着色，请解释为什么填涂颜色超过了四色；
- 3、设计剪枝策略，分析不同的剪枝策略的效率；
- 4、随机产生不同规模的图，分析算法效率与图规模的关系。

三、实验步骤与结果

关于地图数据读取

顶点定义如下，记录了该顶点的所选颜色、对于每种颜色该顶点是否可选、可选颜色的数量和相邻点的数量即顶点的度。

注：0 代表没颜色，颜色从 1 开始，COLOR 为颜色总数。

```
1.  struct Vertex
2.  {
3.      int color; // 该点所选的颜色
4.      int state[COLOR + 1]; // 颜色状态, 1 为可选, 非 1 为不可选
5.      int choice; // 可选颜色的数量, 即 state 中 1 的数量
6.      int degree; // 相邻点的数量, 即该点的度
7.      Vertex() {
8.          color = 0; // 默认该顶点没颜色
9.          for (int i = 0; i <= COLOR; ++i)
10.             state[i] = 1; // 默认初始化为全部颜色都可选
11.          choice = COLOR; // 默认初始化为全部颜色都可选
12.          degree = 0; // 默认初始化为 0
13.      }
14.  };
```

我通过邻接表来储存每个节点的相邻节点信息，包括当前节点的相邻节点总数（也就是度）和相邻节点具体是哪些（即记录当前节点的所有相邻节点编号）。

```
int Map[VERTEX + 1][256];
// 邻接表, Map[i][0] 表示第 i 个节点相邻节点的数量, Map[i][j] 表示第 i 个节点的第 j 个相邻节点。
```

回溯法基础版本

回溯法的基本思想是从问题的初始状态出发，逐步地尝试所有可能的解决方案，当发现当前的解决方案不符合要求时，就回溯到之前的状态，尝试其他可能的解决方案，直到找到符合要求的解决方案或者已经尝试了所有的解决方案。

在地图填色问题中，回溯法具体应用如下：对当前进行填色时，如果颜色合法，则填下一个节点。如果所填颜色非法，则不继续搜索，而是回溯到上一层的节点填别的颜色。

伪代码如下：

```
1.  void DFS(current, count){
2.      if(count == 点总数){
3.          sum += S[current].剩余可选颜色数 // 到达叶子节点, 统计结果
4.      }
```

```

5.     for(i = 1 to 颜色总数){
6.         if(当前颜色 i 可选){
7.             S[current].color=i;
8.         }
9.         if(check(current,i)){ //检查当前节点填颜色 i 是否合法
10.            DFS(下一个节点)
11.            //合法则DFS 继续搜索后再回溯, 不合法就不继续搜索了直接回溯
12.        }
13.        回溯
14.    }
15. }

```

探索如何剪枝

➤ 剪枝策略 1：向前探测

在填涂当前节点颜色时，对当前节点的相邻待填色节点进行遍历，若此节点填当前颜色 i 后会导致其相邻待填色节点中有节点无色可填，那么表明当前节点填颜色 i 是不可行的，是没有前途的。所以我们可以对当前节点填颜色 i 这种情况不继续向下探索，而是直接回溯，这样就做到了剪枝。

下面是一个示例（可供选择的颜色为红黄蓝绿 4 种），观察下图可以发现，在当前已填涂三个区域的情况下，将进行填涂第四个区域的操作，该区域剩余两种颜色可供选择。若选择红色，那么对该区域的相邻节点进行遍历未出现可选颜色为 0 的情况，因此保留；若选择蓝色，那么该区域的相邻节点中就有节点（即圆形区域）会出现可选颜色为 0 的情况，因此对此进行剪枝。

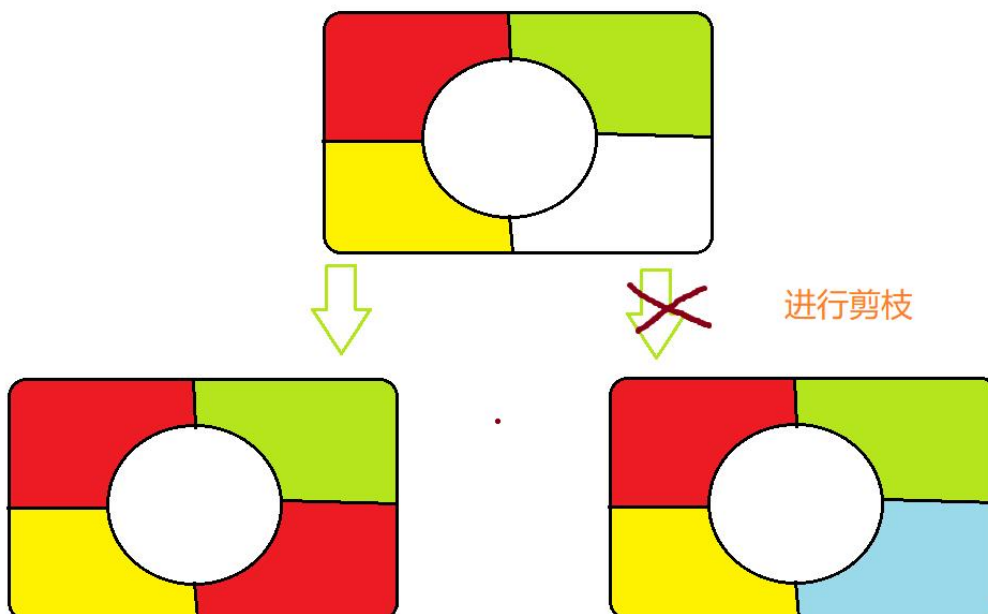


图 1 - 剪枝策略-向前探查法分析例子

根据上面的思路实现的 `check` 函数具体代码如下：

```

1.  bool check(Vertex* S,int current,int i){
2.      //传进节点数组、当前节点序号 current 和其所选的颜色 i
3.      for (int k = 1; k <= Map[current][0]; ++k) { //遍历当前节点的所有相邻节点
4.          int j = Map[current][k]; //j 是遍历到的相邻节点序号
5.          if (S[j].color == 0 && S[j].state[i] == 1) {
6.              //若顶点 j 为待填色节点且可以填颜色 i
7.              S[j].state[i] = -current; //就使得顶点 j 不能选 i 这个颜色
8.              S[j].choice--; //顶点 j 的可选颜色数量-1
9.              if (!S[j].choice ) {
10.                 //若顶点 j 的可选颜色数量-1 之后变为了 0, 返回 false
11.                 return false;
12.             }
13.         }
14.     }
15.     return true; //若不会导致出现相邻顶点无色可选的情况则返回 true
16. }

```

➤ 剪枝策略 2：MRV+DH—选点策略的实现

1. MRV(最少剩余量选择): 在待填色的节点中寻找可选颜色数最小的节点。
2. DH(度最大选择): 在可选颜色数最小的节点中选择度最大的作为下一个需要着色的节点。

为什么实现了剪枝？

MRV: 优先填涂可选颜色数最少的节点，即先对容易导致失败的节点填涂，那么便可以早些发现是否当前填色方案必然失败，从而可以提前回溯，避开不少的不可行解。

DH: 优先对度最多的节点进行填色，假设填色为 c，因为度最多的节点约束了**最多的**其他节点，使得**最多的**其他节点都不会去探索节点填色为 c 的情况（因为不合法），使得**最多的**其他节点的填色为 c 的分枝被剪掉了，从而实现了剪枝。

选点函数实现的具体代码：

```

1.  int getNext(Vertex* S) { //优先 MRH, 其次 DH
2.      auto Min = COLOR; //最小的可选颜色数量, 初始化为最大
3.      int next = 0; //记录下一个需要着色的节点的序号
4.      for (int i = 1; i <= VERTEX; ++i) {
5.          if (!S[i].color ) { //从未填色的点中找到下一个要着色的点
6.              if (S[i].choice == Min) {
7.                  //在可选颜色数最小的节点中选择度最大的作为下一个需要着色的节点
8.                  if (S[i].degree > S[next].degree) {
9.                      Min = S[i].choice;
10.                     next = i;
11.                 }
12.             }
13.             else if (S[i].choice < Min) { //寻找可选颜色数最小的节点

```

```

14.             Min = S[i].choice;
15.             next = i;
16.         }
17.     }
18. }
19. return next; //返回下一个需要着色的节点的序号
20. }

```

❖ 通过小规模数据测试算法正确性

对实验要求给出的小规模数据，利用四色填色测试算法的正确性。

1) 对于该小地图，首先我对各个区域进行编号如下

```

e 1 2
e 1 3
e 1 4
e 2 3
e 2 4
e 2 5
e 3 4
e 4 5
e 4 6
e 4 7
e 5 6
e 5 8
e 6 7
e 6 8
e 6 9
e 7 9
e 8 9

```

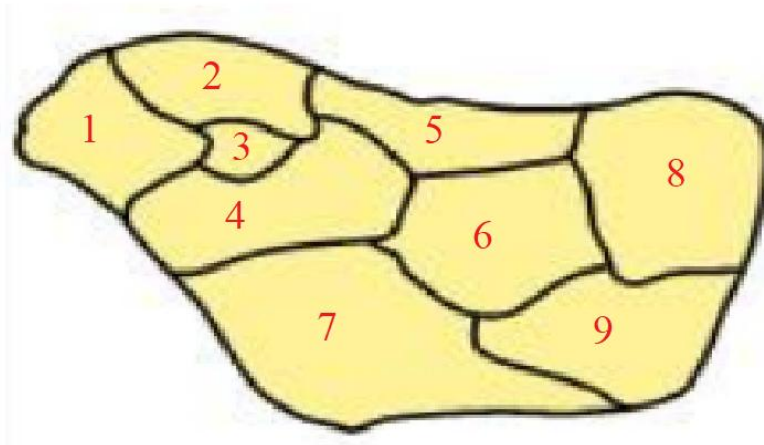


图2 - 对小地图各个区域编号并记录区域间的邻接关系

2) 我利用四色填色测试了算法的正确性，从下图可以看到对于这个小地图，程序在<0.5ms内找到了全部480个解，解的个数正确，算法是正确的。

一、小地图四色测试

=====

顶点数： 9

边数： 17

颜色数： 4

运行时间： 0.131 ms

解的个数： 480

第一个可行解时间： 0.036 ms

是否超时： False

➤ 剪枝策略 3：“树层去重”

前提：每个节点填色时都按颜色编号从小到大尝试填涂(从 1 到 COLOR)

一旦同层节点的某节点填涂了一个之前节点都没用过的新颜色，那么我们在不断深搜计算得到由该节点填涂该新颜色不断拓展最后得到的可行解的数量 n 后，就不再去深搜计算该节点填其他新颜色时的解的数量，因为该节点填其他新颜色时解的数量也为 n ，既然解的个数是一样的，那么就算一次就足够了。统计解总个数的变量 $\text{sum} += n \times \text{新颜色总数}$ 。我们对该树形结构的每一层都做这样的剪枝操作，那么可以想象我们将减去该树形结构大部分的分枝。

我将这种剪枝策略称为“树层去重”。

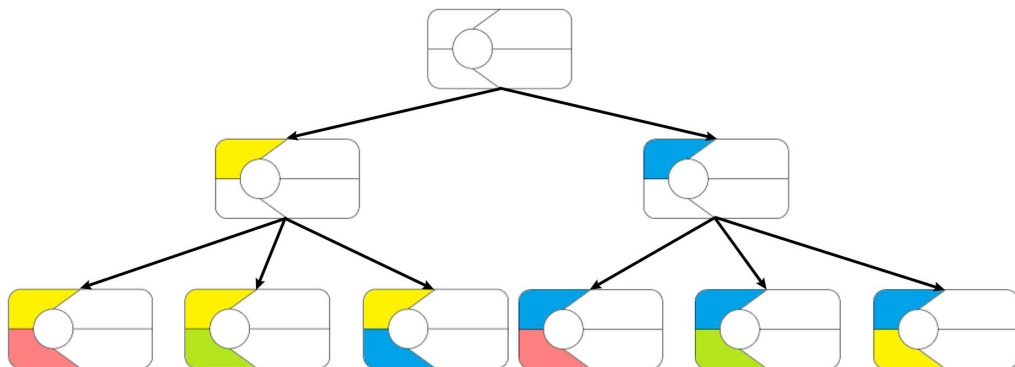


图 3 - 用于讲解“树层去重”的图

剪枝策略总结：

1. 运用邻接表而不是邻接矩阵存储图的信息，这使得在寻找每个节点的相邻节点时不需要遍历整个邻接矩阵而是只需要遍历该节点的相邻节点数组即可。
2. 剪枝策略 1，向前探测——check 函数的实现
3. 剪枝策略 2，MRV+DH——选点策略的实现
4. 剪枝策略 3，“树层去重”的实现

根据实验要求，对附件中给定的地图数据填涂

①得到第一个可行解的时间：

	450 点 5 色	450 点 15 色	450 点 25 色
运行时间	3.190s	5.474s	0.017s

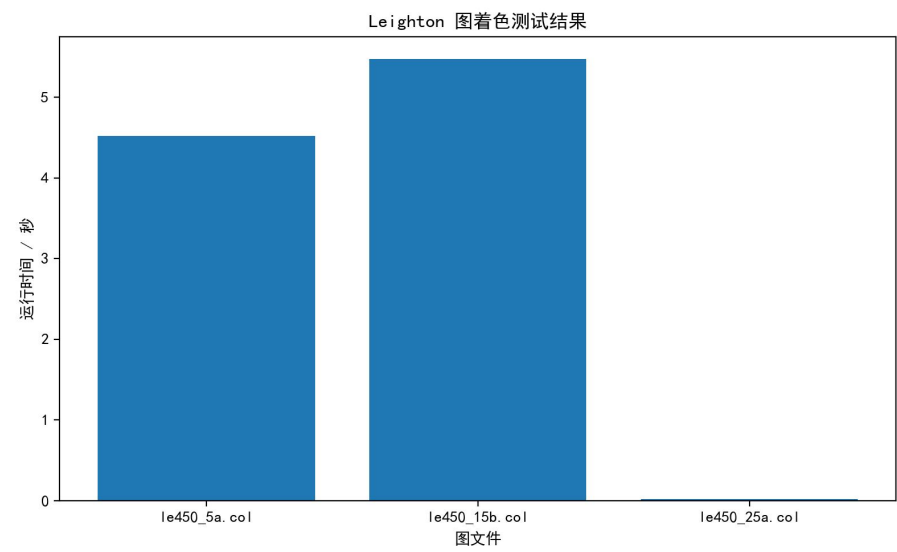
图文件: le450_5a.col
顶点数: 450, 边数: 5714, 颜色数: 5
运行时间: 4.520s
解的个数: 3840
第一个可行解时间: 3.190s
是否超时: False

图文件: le450_15b.col
顶点数: 450, 边数: 8169, 颜色数: 15
运行时间: 5.475s
解的个数: 1
第一个可行解时间: 5.474s
是否超时: False

图文件: le450_25a.col
顶点数: 450, 边数: 8260, 颜色数: 25
运行时间: 0.018s
解的个数: 1
第一个可行解时间: 0.017s
是否超时: False

②尝试得到全部可行解：

1. 对于 450 点 5 色图，可见经过重重优化后仅用 4.520s 就可得到全部的 3840 个解。
2. 对于 450 点 15 色图和 450 点 25 色图，发现无法得到其全部解的个数，久久运行不出来。便只求它的第一个解。



解释为什么填涂颜色超过了四色

根据平面图性质，对于任意平面图，需要满足：

由欧拉公式推导出的： $E \leq 3V - 6$

而实验中的 Leighton 图边数远超该限制，因此它不是平面图，不适用于四色定理，所以可能需要超过 4 种颜色。

❖ 自行生成地图涂色并分析：

(1) 图规模对运行时间的影响：

注：颜色总数均为 4。

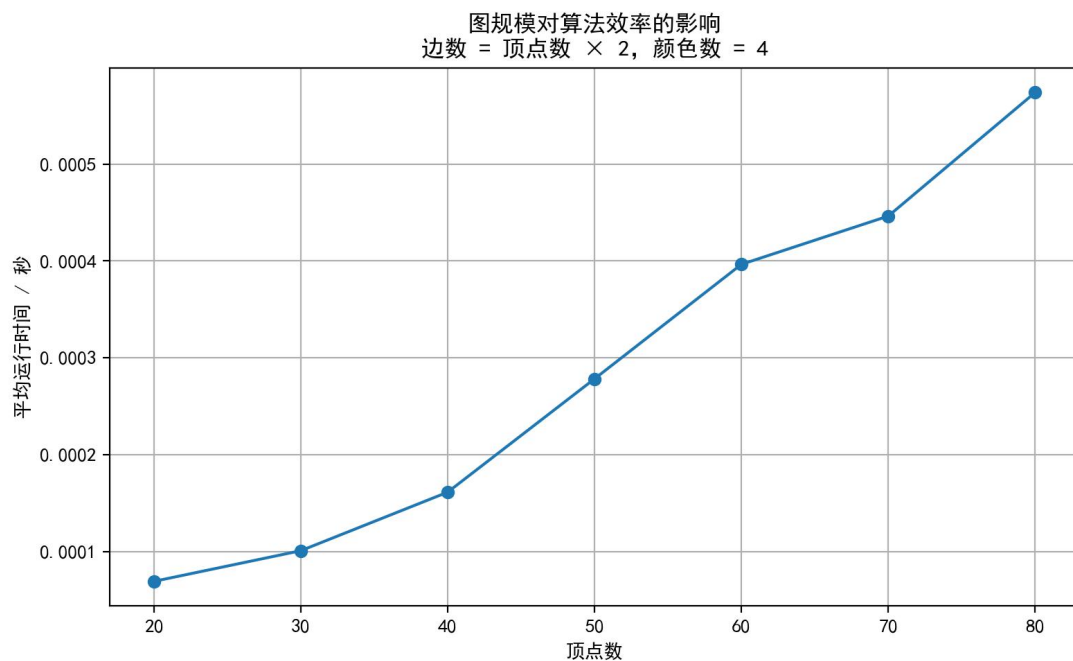


图 4 - 图规模对运行时间的影响

从上面的表格和曲线图可知：随着顶点数增加，算法运行时间呈现出明显的上升趋势。由于图着色问题本质上属于 NP 完全问题，其搜索空间规模随顶点数呈指数级增长，即使采用了剪枝策略，仍无法从根本上改变其复杂度特性。

本实验中，由于边数始终保持为顶点数的两倍，图的密度基本保持稳定，因此问题难度变化不大，运行时间主要受顶点数增长的影响，呈现出较为平滑的上升趋势。

同时，由于实验规模较小，指数增长尚未完全显现，曲线表现为近似线性增长，但随着规模进一步扩大，运行时间将呈现更明显的指数增长特征。

(2) 图的边数对运行时间的影响：

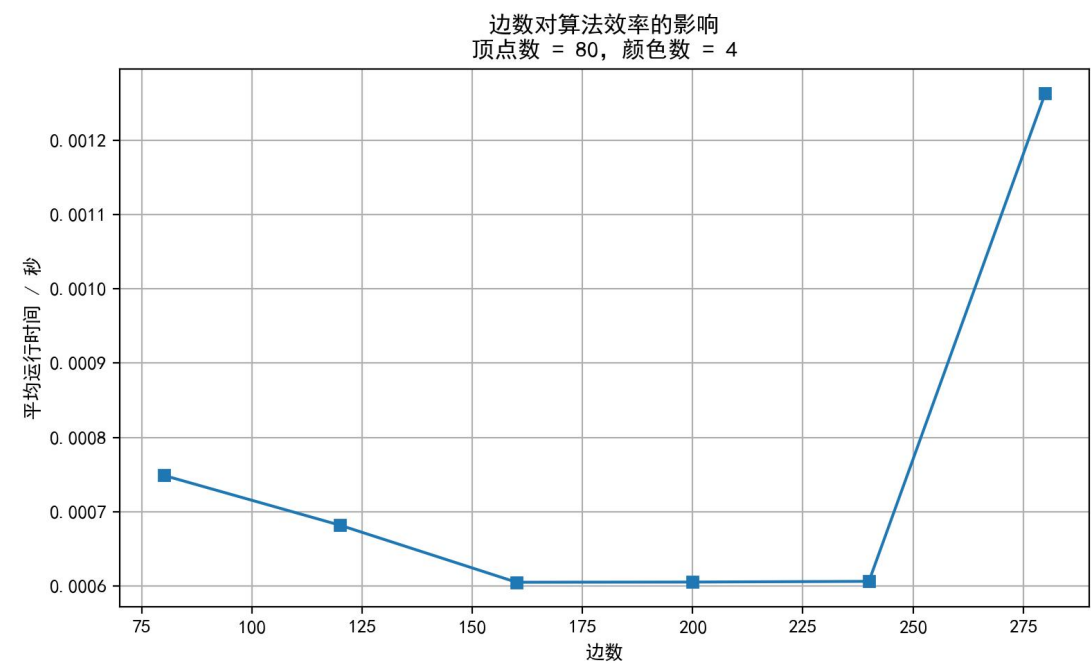


图 5 - 图的边数对运行时间的影响

从上面的表格和曲线图可知：算法运行时间呈现“先下降后上升”的趋势。当边数较少时，图结构较为稀疏，约束较弱，搜索空间较大，导致运行时间较长。随着边数增加，顶点间约束增强，剪枝效果更明显，搜索空间缩小，因此运行时间下降。

但当边数进一步增加时，图变得过于稠密，可行解数量减少甚至无解，算法需要进行大量尝试以验证解的存在性，导致运行时间再次上升。

(3) 颜色数对运行时间的影响：

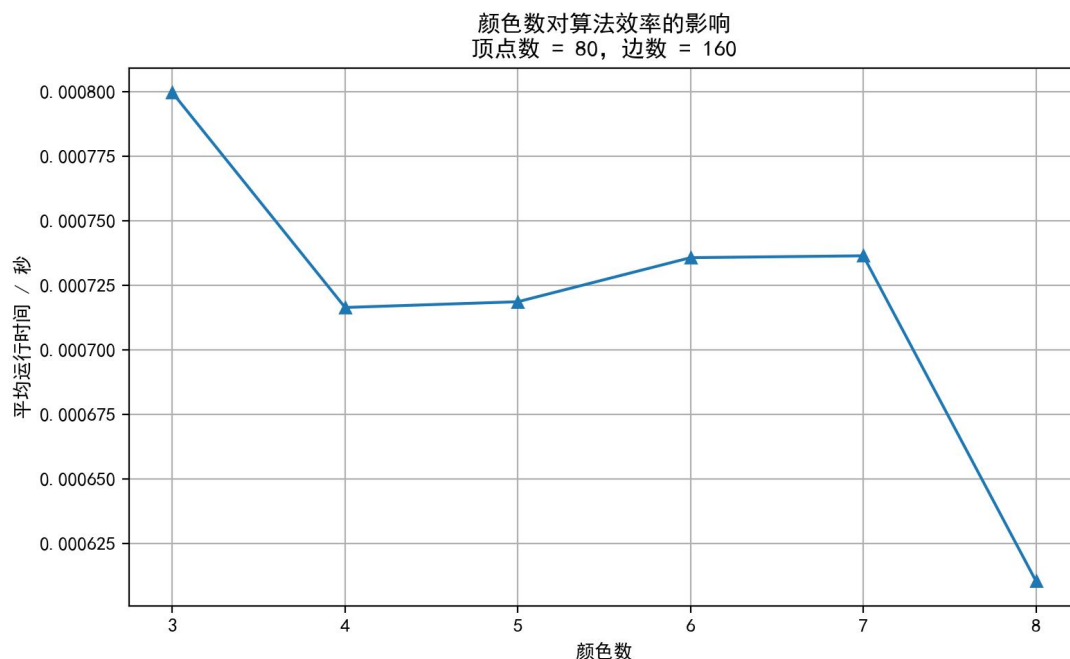


图 6 - 颜色数对运行时间的影响

在颜色数变化实验中，算法运行时间呈现出先下降、后略微上升、再下降的趋势。

当颜色数较少时（如 3 色），由于约束较强，图难以满足着色条件，算法需要进行大量尝试，运行时间较长。

当颜色数增加到 4 时，问题变得更容易满足，算法能够较快找到可行解，运行时间下降。

随着颜色数进一步增加（5~7），虽然问题更容易满足，但每个顶点的可选颜色数增加，搜索分支增多，导致运行时间略有上升。当颜色数较多时（如 8 色），约束显著减弱，冲突概率降低，算法几乎无需回溯即可完成着色，因此运行时间再次下降。

四、实验结论和体会

本实验利用回溯法求解地图填色（图着色）问题，并通过小规模地图验证了算法的正确性。随后，对 Leighton 图数据及随机生成图进行了测试，分析了不同因素对算法效率的影响。

实验结果表明：

随着图规模增大，搜索空间不断扩大，算法运行时间整体呈上升趋势；边数变化时，运行时间呈现“先下降后上升”的现象，这是由于剪枝效果与问题难度共同作用导致的；颜色数较少时约束较强，容易产生大量回溯；颜色数增多后，更

容易找到可行解，但分支数也会增加，因此运行时间呈非单调变化。同时，MRV、DH、Forward Checking 和树层去重等优化策略能够有效减少无效搜索，缩小搜索树规模，提高算法效率。

总体来看，回溯法适用于中小规模图着色问题，但随着问题规模扩大，其指数级复杂度会导致运行时间快速增长，因此需要结合剪枝与启发式策略进行优化。

此外，通过绘制搜索树和性能分析图，我更加理解了不同优化策略对搜索空间的影响，也提高了自己分析算法性能和实验结果的能力。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。